Toast & Notification Pattern

Problem: Build a reusable toast + notification system: `toast.success()/error()/info()/warning()` callable from anywhere, auto-dismiss with pause on hover, a queue with a max number on screen, a promise toast (loading → success/error), an action button (Undo), and a notification center (bell with unread count and history that survives reload).

Tech stack: React (Vite, JavaScript, Tailwind CSS v4)

Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what it must do, what is out of scope. Write the assumptions down.
2. **List the interactions**: every click / key / timer and what should happen. This is the checklist.
3. **Sketch the architecture**: components and modules, one job each.
4. **Design the state**: the minimum source of truth, and who owns it. Everything else is derived.
5. **Define contracts**: data shapes, function signatures, API mocks.
6. **Trace one flow end to end** before writing code.
7. **Find the hard parts** early and decide how to solve them.
8. **Build in layers**: smallest working version first, then add one working layer at a time.
9. **Cover states and edge cases**: empty, loading, error, overflow, rapid clicks.
10. **Wrap up**: explain trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Use cases + architecture + state design	8 min
Store + imperative `toast()` API (no UI polish)	10 min
`Toaster` + `ToastItem` with auto-dismiss	10 min
Pause on hover, queue/max visible, exit animation	8 min
Promise toast + action button	6 min
Notification center (history, unread, persist)	8 min
Accessibility pass, edge cases, wrap-up	5 min

Think out loud

Say every decision and why while you type: *"I'm keeping the store outside React so an axios interceptor can call `toast.error()` without a hook."* The interviewer grades your reasoning as much as the result. If you get stuck, say what you are trying and what you will fall back to.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show a toast with a **type**: `success`, `error`, `warning`, `info`, `loading`.
- **Imperative API** callable from anywhere: `toast.success('Saved')`, not only from components.
- **Auto-dismiss** after a duration (per type; errors stay longer, loading never auto-dismisses).
- **Pause** the timer while hovered or focused; resume with the remaining time.
- **Manual dismiss**: close button, `Escape` on a focused toast, `toast.dismiss(id)` and `toast.dismiss()` for all.
- **Queue**: at most N toasts on screen; the rest wait and show as slots free up ("+3 more").
- **Promise toast**: one toast that goes `loading → success` or `loading → error` in place.
- **Action button** (e.g. *Undo*) that runs a callback and closes the toast.
- **Notification center**: bell with unread badge, panel with history, mark one / all read, clear; history survives reload.
- Configurable **position** (top/bottom, left/right).

Non-functional requirements

Area	Requirement
Performance	Only the toast list re-renders, not the whole app. One timer per *visible* toast. Progress bar animated with CSS `transform`, not JS ticks.
Scalability	A burst of 100 toasts must not flood the screen: queue + cap. History capped at 50.
Error handling	`localStorage` can throw (quota, private mode): wrap reads/writes; the app keeps working.
Accessibility	Live region (`aria-live="polite"`), errors use `role="alert"`, close button has `aria-label`, pause on focus (WCAG 2.2.1 Timing Adjustable), reduced motion respected.
Reusability	Zero coupling to app: drop `<Toaster />` once, call `toast()` anywhere.
Extensibility	New type = one entry in `DEFAULT_DURATION` and `STYLES`. Custom content / positions without touching the store.

Clarifying questions to ask the interviewer

1. Should `toast()` be callable **outside React** (API layer, interceptors), or is a `useToast()` hook enough?
2. How many toasts can be on screen at once, and what happens to the extra ones: **queue** or **drop the oldest**?
3. Is the **notification center** the same data as toasts (history of toasts) or a separate server feed?
4. Should history **persist** across reloads? Across tabs? Per user (server)?
5. Do we need **promise** and **action (Undo)** toasts, or only static messages?
6. Any animation / position requirements, and must we respect `prefers-reduced-motion`?

Assumptions we will build with

- `toast()` is a plain function backed by a **module-level store**; React subscribes with `useSyncExternalStore`.
- Max **3** on screen, **FIFO queue** for the rest. A queued toast's timer starts only when it becomes visible.
- Durations: success/info 4s, warning 5s, error 6s, loading ∞. A caller can override `duration` (`Infinity` = sticky).
- Every **final** toast (not `loading`) is also written to the notification history (newest first, max 50), persisted in `localStorage`.
- Same `id` = **update in place** (this is how the promise toast works and how callers dedupe).
- No backend; one mock API (`fakeSave`) to demo the promise toast.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
`toast.success('Profile saved')`	Green toast slides in, progress bar shrinks, disappears after 4s
`toast.error('Payment failed')`	Red toast with `role="alert"` (announced immediately), stays 6s
Hover (or Tab into) a toast	Timer and progress bar pause
Mouse leaves / focus leaves	Timer resumes with the **remaining** time, not a full reset
Click × / press `Escape` on a focused toast	Exit animation, then removed
Fire 6 toasts quickly	3 visible, "+3 more" shown; each time one closes, the next appears with a fresh timer
`toast.dismiss()`	All toasts (visible and queued) animate out and are removed
`toast.promise(save(), {...})`	One spinner toast "Saving…"; on resolve it becomes success, on reject error, **same toast**
`toast('Item deleted', { action: { label: 'Undo', onClick } })`	Toast shows Undo button; click runs `onClick` and closes the toast
Call `toast.success('x', { id: 'sync' })` twice	One toast updated, not two

Action / input	Expected behavior / output
Any final toast	Added to the notification history as unread; bell badge count goes up
Click the bell	Panel opens with history, newest first, unread highlighted
Click a history item	That item is marked read
"Mark all read" / "Clear"	Badge goes to 0 / list empties ("You're all caught up")
Click outside panel / `Escape`	Panel closes
Reload page	History and unread state are still there; on-screen toasts are not (they are transient)
`localStorage` full / blocked	Toasts still work, history just isn't persisted

3. Design the architecture

Module	Responsibility
`toastStore.js`	Single source of truth (`toasts` , `notifications`), `subscribe/getSnapshot` , actions, pure helpers, persistence
`toast.js`	Public imperative API: `toast()` , `.success/.error/.info/.warning/.loading` , `.promise` , `.dismiss`
`useToastStore.js`	Bridges the store into React with `useSyncExternalStore`
`Toaster.jsx`	Mounted once; live region; takes the first `max` toasts, shows "+N more"
`ToastItem.jsx`	One toast: its own timer, pause/resume, progress bar, close/action/Escape, enter/exit animation
`NotificationCenter.jsx`	Bell + unread badge + panel (mark read, clear, outside click)
`timeAgo.js`	Pure "2m ago" formatter
`api/fakeSave.js`	Mock async call to demo the promise toast

```
App
├── header
│   └── NotificationCenter ──reads──► store.notifications
├── main (demo buttons) ──calls──► toast.*()  ──► toastStore actions
└── Toaster ──reads──► store.toasts
    └── ToastItem × max  ──calls──► dismissToast(id)

toast.js ──► toastStore.js ◄── useToastStore (useSyncExternalStore)
                └── localStorage (notifications only)
```

Why a store outside React and not Context? Context forces callers to be components (`useToast()`), and the most common caller of an error toast is *not* a component: an API interceptor, a `catch` in a util, a

websocket handler. A tiny pub/sub store + `useSyncExternalStore` gives both: call from anywhere, render in React, and no tearing in concurrent rendering. Start simple: one store file; split only if it grows.

---

4. Design the state / data model

State or field	Purpose
`toasts: Toast[]`	All live toasts in FIFO order (visible + queued). Owned by the store.
`toast.id`	Stable key; same id = update in place
`toast.type`	Style, icon, ARIA role, default duration
`toast.message`	Text
`toast.duration`	ms until auto-dismiss; `Infinity` = sticky
`toast.action?`	`{ label, onClick }` for Undo-style buttons
`toast.leaving`	`true` during the 200ms exit animation
`notifications: Notification[]`	History, newest first, max 50. Owned by the store, persisted.
`notification.read`	Unread vs read
`paused` (in `ToastItem`)	UI-only: hovered/focused. Local `useState`.
`remaining` (ref in `ToastItem`)	ms left on the timer. A `ref` because changing it must not re-render.
`open` (in `NotificationCenter`)	Panel open/closed. Local `useState`.

```
// store state
let state = {
  toasts: [
    { id: 't1', type: 'success', message: 'Profile saved', duration: 4000, action: undefined, leaving:
false },
  ],
  notifications: [
    { id: 't1:success', type: 'success', message: 'Profile saved', createdAt: 1730540000000, read:
false },
  ],
};
```

Senior note: derive, don't store

- **Visible toasts** = `toasts.slice(0, max)`. **Queued count** = `toasts.length - max`. No separate `queue` array to keep in sync.
- **Unread count** = `notifications.filter(n => !n.read).length`. Never a counter that can drift.
- **Timer state lives in the component**, not the store: the store holds data, `ToastItem` owns "time". That also means queued toasts have no timer at all: their clock starts when they mount.

- **Ownership:** store owns everything shared (`toasts` , `notifications`); components own only UI state (`paused` , `open`).

---

5. Design the data structures and contracts

Example data

```
const toasts = [
  { id: 't7', type: 'loading', message: 'Saving report…', duration: Infinity, leaving: false },
  { id: 't8', type: 'info', message: 'Item deleted', duration: 4000, leaving: false,
    action: { label: 'Undo', onClick: () => restoreItem(42) } },
];
```

Stable ids matter twice: React `key` (so a toast doesn't remount and replay its animation when another one leaves), and **update-in-place** (the promise toast reuses its id). The notification id is `${toastId}:${type}` , so updating the same toast to the same final type twice cannot create duplicate history rows.

Public API contract

Call	Input	Returns
`toast(message, options?)`	`options = { id?, type?, duration?, action? }`	`id` (string)
`toast.success / error / info / warning / loading(message, options?)`	same, `type` fixed	`id`
`toast.promise(promise, { loading, success, error })`	`success` / `error` are strings or `(value) => string`	the same `promise` (so callers can still `await` it)
`toast.dismiss(id?)`	no id = all	`void`
`<Toaster position? max? />`	`'top-right' | 'top-left' | 'bottom-right' | 'bottom-left'` , default `bottom-right` , `max = 3`	element
`<NotificationCenter />`	none	element

Mock API

```
// fakeSave(): Promise<{ name: string }>, resolves after 1.5s, rejects ~30% with Error('Network error, save failed')
toast.promise(fakeSave(), {
  loading: 'Saving report…',
  success: (file) => `Saved ${file.name}`,
  error: (err) => err.message,
});
```

Storage contract

`localStorage['notifications'] = JSON.stringify(Notification[])`: only plain data (no `action` functions), at most 50 rows.

---

6. Trace the data flow

Promise toast, end to end:

```
User clicks "Promise (save)"
  │
  ▼
toast.promise(fakeSave(), msgs)
  │  toast.loading('Saving report…')  → showToast({ type:'loading' }) → id 't7'
  ▼
store: toasts = upsertToast(toasts, {id:'t7', type:'loading', duration:Infinity})
       notifications unchanged (loading is not history)
  │  setState → listeners() → useSyncExternalStore sees a new snapshot
  ▼
Toaster re-renders → slice(0, 3) → <ToastItem key="t7"> mounts (spinner, no timer: duration = ∞)
  │
  ▼  … 1.5s later fakeSave resolves { name: 'report.pdf' }
toast.success('Saved report.pdf', { id: 't7' })
  ▼
store: upsertToast merges into t7 → type 'success', duration 4000
       addNotification → history row 't7:success' (unread) → persisted to localStorage
  ▼
ToastItem t7 re-renders (same key, no remount): green icon, progress bar appears
  effect([duration]) resets remaining = 4000 → timer effect starts setTimeout(4000)
NotificationCenter badge: unread + 1
  ▼  4s later (paused time not counted)
dismissToast('t7') → leaving: true → exit animation (200ms) → removed from toasts
  ▼
Toaster re-renders → next queued toast (if any) becomes visible and starts its own timer
```

7. Important technical challenges

Challenge 1: Calling `toast()` from anywhere (outside React)

Problem: An axios interceptor gets a 500. It needs to show an error toast, but it is not a component, so it cannot call `useContext`. A Context-only design forces you to pass `showToast` down or hack a global ref.

Solution: A tiny external store (pub/sub) with `subscribe` + `getSnapshot`; React reads it with `useSyncExternalStore`, which is built for exactly this and is safe under concurrent rendering (no tearing). The store replaces `state` immutably, so the snapshot reference changes only on real updates.

```
let state = { toasts: [], notifications: loadNotifications() };
const listeners = new Set();
function setState(next) {
  state = { ...state, ...next };
  listeners.forEach((l) => l());
}
export const subscribe = (l) => { listeners.add(l); return () => listeners.delete(l); };
export const getSnapshot = () => state;

// React side
export const useToastStore = () => useSyncExternalStore(subscribe, getSnapshot);

// anywhere
api.interceptors.response.use(null, (err) => { toast.error(err.message); throw err; });
```

Challenge 2: Pause on hover and resume with the *remaining* time

Problem: A 6s error toast; the user hovers at 4s to read it, leaves after 10s. A naive `setTimeout(dismiss, duration)` either ignores hover (toast vanishes under the cursor) or restarts at a full 6s on leave.

Solution: Keep `remaining` in a **ref**. Each "running" period starts a timer for `remaining`; the effect cleanup (pause, unmount or dependency change) clears it and subtracts the elapsed time. The progress bar is a CSS animation whose `animation-play-state` follows the same `paused` flag, so they stay in sync without JS ticks.

```
const [paused, setPaused] = useState(false);
const remaining = useRef(duration);

useEffect(() => { remaining.current = duration; }, [duration]); // loading → success: new clock

useEffect(() => {
  if (paused || leaving || duration === Infinity) return;
  const start = Date.now();
  const timer = setTimeout(() => dismissToast(id), remaining.current);
  return () => { clearTimeout(timer); remaining.current -= Date.now() - start; };
}, [id, paused, leaving, duration]);
```

Pause on **focus** too (`onFocus` / `onBlur` bubble in React), so keyboard and screen-reader users get the same time to read and to reach the Undo button.

Challenge 3: Queue + update-in-place (promise toast, dedupe)

Problem: A bulk action fires 30 toasts → wall of toasts covering the UI. And `toast.promise` must not show "Saving…" *and then* a second "Saved" toast.

Solution: One ordered array; visibility is **derived** (`slice(0, max)`), so the queue needs no extra state, and a queued toast has no timer until it mounts. A pure `upsertToast` decides append vs merge by `id`: the promise toast reuses its id, and callers can pass their own id to dedupe ("Reconnecting…" fired 10 times = 1 toast).

```
export function upsertToast(toasts, toast) {
  const i = toasts.findIndex((t) => t.id === toast.id);
  if (i === -1) return [...toasts, toast];
  const next = [...toasts];
  next[i] = { ...toasts[i], ...toast };
  return next;
}

// Toaster
const visible = toasts.slice(0, max);
const queued = toasts.length - visible.length;
```

Challenge 4: Exit animation + accessible announcements

Problem: If we remove the toast from state immediately, React unmounts it and there is nothing left to animate. And a live region that is inserted *together with* its content is often not announced by screen readers.

Solution: Two-phase dismiss: mark `leaving: true` (plays `animate-toast-out`), remove after `EXIT_MS`. The `Toaster` `<section aria-live="polite">` is **always mounted**, so new children are announced; errors use `role="alert"` (assertive). `prefers-reduced-motion` turns the slide off.

```
export function dismissToast(id) {
  const match = (t) => id === undefined || t.id === id;
  setState({ toasts: state.toasts.map((t) => (match(t) ? { ...t, leaving: true } : t)) });
  setTimeout(() => setState({ toasts: state.toasts.filter((t) => !(t.leaving && match(t))) }),
EXIT_MS);
}
```

8. Implement in the correct order

#	Step	Working result
1	`toastStore.js`: `state`, `subscribe`, `getSnapshot`, `showToast` (append only)	Store works in a node test
2	`useToastStore` + `Toaster` rendering plain `<p>` per toast	Button → text appears
3	`toast.js` imperative API with typed helpers	`toast.success()` from anywhere
4	`ToastItem` with styles per type + close button + `dismissToast` (instant remove)	Toasts look right, closable
5	Auto-dismiss timer per toast (fixed duration)	Toasts disappear on their own
6	Pause/resume with `remaining` ref + progress bar `animation-play-state`	Hover/focus pauses
7	`max` + derived queue + "+N more"	Burst of 6 behaves
8	Two-phase dismiss (`leaving`) + enter/exit keyframes + reduced motion	Smooth in/out
9	`upsertToast` + `toast.promise` + `action` button	Promise and Undo work
10	`notifications` in store + `addNotification` + `localStorage`	History persists
11	`NotificationCenter`: badge, panel, mark read, clear, outside click / Escape	Full feature
12	A11y pass (roles, labels, focus pause) + edge cases + positions	Done

9. Cover all states and edge cases

State / edge case	Handling
Initial (no toasts)	Empty live region stays mounted (needed for announcements)
Loading toast	Spinner, `duration = Infinity`, no progress bar, never auto-dismisses
Success / info / warning	Auto-dismiss 4s / 4s / 5s with progress bar
Error	`role="alert"`, 6s, red
Hovered / focused	Timer + bar paused; resume with remaining time

State / edge case	Handling
More toasts than `max`	Extra wait in queue, "+N more" shown, timer starts only when visible
Same id fired again	Updated in place, no duplicate (and no duplicate history row)
Promise resolves / rejects	Same toast becomes success / error with message from value / error
User closes the loading toast, promise settles later	Final toast is shown again with the same id (the result is never lost)
Dismiss during exit animation / dismiss twice	Idempotent: still `leaving`, removed once
`toast.dismiss()` with queued toasts	Queued ones are cleared too
Action (Undo) clicked	Callback runs, toast dismissed
Very long message	Wraps; toast width capped (`w-80`, `max-w-[calc(100vw-2rem)]`)
Mobile width	Width shrinks to viewport minus gutter
History empty	"You're all caught up"
Unread > 99	Badge shows "99+"
History > 50	Oldest dropped (`slice(0, 50)`)
`localStorage` corrupted / blocked / full	`try/catch` → start empty / skip persisting; toasts still work
`prefers-reduced-motion`	Slide animations disabled
React StrictMode double effects	Timer effect cleanup subtracts ~0ms and restarts: no double dismiss

10. Final structure

```
src/
├── api/
│   └── fakeSave.js            # mock async call for the promise toast
├── toast/
│   ├── toastStore.js          # store, pure helpers, actions, persistence
│   ├── toast.js               # public imperative API
│   ├── useToastStore.js       # useSyncExternalStore bridge
│   ├── Toaster.jsx            # live region, max visible, queue count
│   └── ToastItem.jsx          # one toast: timer, pause, progress, actions
├── notifications/
│   ├── NotificationCenter.jsx # bell, badge, panel
│   └── timeAgo.js             # "2m ago"
├── App.jsx                    # demo page
├── main.jsx
└── index.css                  # Tailwind + keyframes
```

11. Full working code

Setup: Vite React (SWC) template + `npm i tailwindcss @tailwindcss/vite`.

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

`src/index.css`

```
@import "tailwindcss";

@theme {
  --animate-toast-in: toast-in 200ms ease-out;
  --animate-toast-out: toast-out 200ms ease-in forwards;

  @keyframes toast-in {
    from { opacity: 0; transform: translateX(100%); }
    to { opacity: 1; transform: none; }
  }
  @keyframes toast-out {
    to { opacity: 0; transform: translateX(100%); }
  }
}

@keyframes toast-progress {
  from { transform: scaleX(1); }
  to { transform: scaleX(0); }
}

@media (prefers-reduced-motion: reduce) {
  .animate-toast-in,
  .animate-toast-out {
    animation: none;
  }
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

`src/toast/toastStore.js`

```
// Global store that lives outside React, so toast() can be called from anywhere
// (event handlers, api layer, utils) without a Provider or a hook.

export const DEFAULT_DURATION = { success: 4000, info: 4000, warning: 5000, error: 6000, loading:
Infinity };
export const EXIT_MS = 200;
const STORAGE_KEY = 'notifications';
const MAX_HISTORY = 50;

function loadNotifications() {
  try {
    return JSON.parse(localStorage.getItem(STORAGE_KEY)) ?? [];
  } catch {
    return [];
  }
}

let state = { toasts: [], notifications: loadNotifications() };
const listeners = new Set();

function setState(next) {
  const prev = state;
  state = { ...state, ...next };
  if (state.notifications !== prev.notifications) {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(state.notifications));
    } catch {
      // quota exceeded / private mode: history just won't survive a reload
    }
  }
  listeners.forEach((listener) => listener());
}

export const subscribe = (listener) => {
  listeners.add(listener);
  return () => listeners.delete(listener);
};
export const getSnapshot = () => state;

// ---------- pure helpers (unit-testable) ----------

// Same id = update in place (promise toast: loading -> success), new id = append to the queue.
export function upsertToast(toasts, toast) {
  const i = toasts.findIndex((t) => t.id === toast.id);
  if (i === -1) return [...toasts, toast];
```

```
  const next = [...toasts];
  next[i] = { ...toasts[i], ...toast };
  return next;
}

// Only final states go to history; id:type makes a repeated update idempotent.
export function addNotification(list, toast, max = MAX_HISTORY) {
  if (toast.type === 'loading') return list;
  const id = `${toast.id}:${toast.type}`;
  if (list.some((n) => n.id === id)) return list;
  const item = { id, type: toast.type, message: toast.message, createdAt: Date.now(), read: false };
  return [item, ...list].slice(0, max);
}

// ---------- actions ----------

let seq = 0;

export function showToast({ id = `t${++seq}`, type = 'info', message, duration, action }) {
  const toast = { id, type, message, duration: duration ?? DEFAULT_DURATION[type], action, leaving:
false };
  setState({
    toasts: upsertToast(state.toasts, toast),
    notifications: addNotification(state.notifications, toast),
  });
  return id;
}

// No id = dismiss all. The toast stays in the list for EXIT_MS so the exit animation can play.
export function dismissToast(id) {
  const match = (t) => id === undefined || t.id === id;
  setState({ toasts: state.toasts.map((t) => (match(t) ? { ...t, leaving: true } : t)) });
  setTimeout(() => {
    setState({ toasts: state.toasts.filter((t) => !(t.leaving && match(t))) });
  }, EXIT_MS);
}

export const markRead = (id) =>
  setState({ notifications: state.notifications.map((n) => (n.id === id ? { ...n, read: true } : n))
});

export const markAllRead = () =>
  setState({ notifications: state.notifications.map((n) => (n.read ? n : { ...n, read: true })) });

export const clearNotifications = () => setState({ notifications: [] });
```

`src/toast/toast.js`

```
import { showToast, dismissToast } from './toastStore.js';

// Public, imperative API: toast('Hi'), toast.success('Saved'), toast.promise(p, msgs),
toast.dismiss(id)
export function toast(message, options = {}) {
  return showToast({ ...options, message });
}

toast.success = (message, options) => toast(message, { ...options, type: 'success' });
toast.error = (message, options) => toast(message, { ...options, type: 'error' });
toast.info = (message, options) => toast(message, { ...options, type: 'info' });
toast.warning = (message, options) => toast(message, { ...options, type: 'warning' });
toast.loading = (message, options) => toast(message, { ...options, type: 'loading' });
toast.dismiss = dismissToast;

// One toast for the whole async lifecycle: same id, so it is updated in place, not stacked.
toast.promise = (promise, { loading, success, error }) => {
  const id = toast.loading(loading);
  promise.then(
    (data) => toast.success(typeof success === 'function' ? success(data) : success, { id }),
    (err) => toast.error(typeof error === 'function' ? error(err) : error, { id }),
  );
  return promise;
};
```

`src/toast/useToastStore.js`

```
import { useSyncExternalStore } from 'react';
import { subscribe, getSnapshot } from './toastStore';

export const useToastStore = () => useSyncExternalStore(subscribe, getSnapshot);
```

`src/toast/Toaster.jsx`

```
import { useToastStore } from './useToastStore';
import ToastItem from './ToastItem';

const POSITIONS = {
  'top-right': 'top-4 right-4',
  'top-left': 'top-4 left-4',
  'bottom-right': 'bottom-4 right-4',
  'bottom-left': 'bottom-4 left-4',
};

export default function Toaster({ position = 'bottom-right', max = 3 }) {
  const { toasts } = useToastStore();
  // Queue is derived: first `max` are on screen, the rest wait (and their timers haven't started).
  const visible = toasts.slice(0, max);
  const queued = toasts.length - visible.length;

  return (
    // Always mounted, so screen readers already watch the live region when a toast is inserted.
    <section
      aria-label="Notifications"
      aria-live="polite"
      className={`fixed z-50 flex w-80 max-w-[calc(100vw-2rem)] flex-col gap-2 ${POSITIONS[position]}`}
    >
      {visible.map((t) => (
        <ToastItem key={t.id} toast={t} />
      ))}
      {queued > 0 && <p className="text-center text-xs text-slate-500">+{queued} more</p>}
    </section>
  );
}
```

`src/toast/ToastItem.jsx`

```
import { useEffect, useRef, useState } from 'react';
import { dismissToast } from './toastStore';

const STYLES = {
  success: { icon: '✓', badge: 'bg-green-500', border: 'border-green-500', bar: 'bg-green-500' },
  error: { icon: '✕', badge: 'bg-red-500', border: 'border-red-500', bar: 'bg-red-500' },
  warning: { icon: '!', badge: 'bg-amber-500', border: 'border-amber-500', bar: 'bg-amber-500' },
  info: { icon: 'i', badge: 'bg-blue-500', border: 'border-blue-500', bar: 'bg-blue-500' },
  loading: { icon: '', badge: 'animate-spin border-2 border-slate-300 border-t-slate-600', border:
'border-slate-400', bar: '' },
};

export default function ToastItem({ toast }) {
  const { id, type, message, duration, action, leaving } = toast;
  const [paused, setPaused] = useState(false);
  const remaining = useRef(duration);

  // A promise toast goes loading (Infinity) -> success (4000): restart the clock with the new
duration.
  useEffect(() => {
    remaining.current = duration;
  }, [duration]);

  // Pause = clear the timer and remember how much time was left; resume = new timer for the rest.
  useEffect(() => {
    if (paused || leaving || duration === Infinity) return;
    const start = Date.now();
    const timer = setTimeout(() => dismissToast(id), remaining.current);
    return () => {
      clearTimeout(timer);
      remaining.current -= Date.now() - start;
    };
  }, [id, paused, leaving, duration]);

  const s = STYLES[type];

  return (
    <div
      role={type === 'error' ? 'alert' : 'status'}
      onMouseEnter={() => setPaused(true)}
      onMouseLeave={() => setPaused(false)}
      onFocus={() => setPaused(true)}
      onBlur={() => setPaused(false)}
      onKeyDown={(e) => e.key === 'Escape' && dismissToast(id)}
      className={`relative overflow-hidden rounded-lg border-l-4 bg-white p-3 pr-9 shadow-lg
```

```
${s.border} ${
        leaving ? 'animate-toast-out' : 'animate-toast-in'
      }`}
    >
      <div className="flex items-start gap-2">
        <span
          aria-hidden="true"
          className={`grid h-5 w-5 shrink-0 place-items-center rounded-full text-xs font-bold text-
white ${s.badge}`}
        >
          {s.icon}
        </span>
        <p className="text-sm text-slate-800">{message}</p>
      </div>

      {action && (
        <button
          onClick={() => {
            action.onClick();
            dismissToast(id);
          }}
          className="ml-7 mt-1 text-sm font-medium text-blue-600 hover:underline"
        >
          {action.label}
        </button>
      )}

      <button
        aria-label="Dismiss notification"
        onClick={() => dismissToast(id)}
        className="absolute right-2 top-2 rounded px-1.5 text-slate-400 hover:bg-slate-100 hover:text-
slate-700"
      >
        ×
      </button>

      {duration !== Infinity && (
        <div
          aria-hidden="true"
          className={`absolute bottom-0 left-0 h-1 w-full origin-left ${s.bar}`}
          style={{
            animationName: 'toast-progress',
            animationDuration: `${duration}ms`,
            animationTimingFunction: 'linear',
            animationFillMode: 'forwards',
            animationPlayState: paused ? 'paused' : 'running',
          }}
```

```
        />
      )}
    </div>
  );
}
```

src/notifications/timeAgo.js

```
export function timeAgo(timestamp, now = Date.now()) {
  const sec = Math.floor((now - timestamp) / 1000);
  if (sec < 60) return 'just now';
  if (sec < 3600) return `${Math.floor(sec / 60)}m ago`;
  if (sec < 86400) return `${Math.floor(sec / 3600)}h ago`;
  return `${Math.floor(sec / 86400)}d ago`;
}
```

`src/notifications/NotificationCenter.jsx`

```
import { useEffect, useRef, useState } from 'react';
import { useToastStore } from '../toast/useToastStore';
import { markRead, markAllRead, clearNotifications } from '../toast/toastStore';
import { timeAgo } from './timeAgo';

const DOT = { success: 'bg-green-500', error: 'bg-red-500', warning: 'bg-amber-500', info: 'bg-blue-
500' };

export default function NotificationCenter() {
  const { notifications } = useToastStore();
  const [open, setOpen] = useState(false);
  const rootRef = useRef(null);
  const unread = notifications.filter((n) => !n.read).length; // derived, never stored

  // Close on outside click / Escape, only while open.
  useEffect(() => {
    if (!open) return;
    const onPointerDown = (e) => !rootRef.current.contains(e.target) && setOpen(false);
    const onKeyDown = (e) => e.key === 'Escape' && setOpen(false);
    document.addEventListener('pointerdown', onPointerDown);
    document.addEventListener('keydown', onKeyDown);
    return () => {
      document.removeEventListener('pointerdown', onPointerDown);
      document.removeEventListener('keydown', onKeyDown);
    };
  }, [open]);

  return (
    <div ref={rootRef} className="relative">
      <button
        aria-label={`Notifications, ${unread} unread`}
        aria-expanded={open}
        aria-controls="notification-panel"
        onClick={() => setOpen((o) => !o)}
        className="relative rounded-full p-2 hover:bg-slate-100"
      >
        <span aria-hidden="true" className="text-xl">
          🔔
        </span>
        {unread > 0 && (
          <span className="absolute -right-0.5 -top-0.5 min-w-5 rounded-full bg-red-500 px-1 text-
center text-xs font-semibold text-white">
            {unread > 99 ? '99+' : unread}
          </span>
        )}
```

```
      </button>

      {open && (
        <div
          id="notification-panel"
          className="absolute right-0 z-40 mt-2 w-80 overflow-hidden rounded-lg border border-slate-200
bg-white shadow-xl"
        >
          <header className="flex items-center justify-between border-b border-slate-200 p-3">
            <h2 className="font-semibold">Notifications</h2>
            <div className="flex gap-3 text-xs">
              <button onClick={markAllRead} disabled={!unread} className="text-blue-600 disabled:text-
slate-400">
                Mark all read
              </button>
              <button
                onClick={clearNotifications}
                disabled={!notifications.length}
                className="text-red-600 disabled:text-slate-400"
              >
                Clear
              </button>
            </div>
          </header>

          {notifications.length === 0 ? (
            <p className="p-6 text-center text-sm text-slate-500">You're all caught up</p>
          ) : (
            <ul className="max-h-80 overflow-y-auto">
              {notifications.map((n) => (
                <li key={n.id}>
                  <button
                    onClick={() => markRead(n.id)}
                    className={`flex w-full items-start gap-2 border-b border-slate-100 p-3 text-left
text-sm hover:bg-slate-50 ${
                      n.read ? 'text-slate-500' : 'bg-blue-50 text-slate-800'
                    }`}
                  >
                    <span aria-hidden="true" className={`mt-1.5 h-2 w-2 shrink-0 rounded-full
${DOT[n.type]}`} />
                    <span className="flex-1">
                      {n.message}
                      {!n.read && <span className="sr-only"> (unread)</span>}
                    </span>
                    <time className="shrink-0 text-xs text-slate-400" dateTime={new
Date(n.createdAt).toISOString()}>
                      {timeAgo(n.createdAt)}
```

```
                  </time>
                </button>
              </li>
            ))}
          </ul>
        )}
      </div>
    )}
  </div>
  );
}
```

src/api/fakeSave.js

```
// Mock API: resolves after 1.5s, fails ~30% of the time so both promise paths show up.
export function fakeSave() {
  return new Promise((resolve, reject) =>
    setTimeout(
      () => (Math.random() < 0.7 ? resolve({ name: 'report.pdf' }) : reject(new Error('Network error,
save failed'))),
      1500,
    ),
  );
}
```

`src/App.jsx`

```
import Toaster from './toast/Toaster';
import { toast } from './toast/toast';
import NotificationCenter from './notifications/NotificationCenter';
import { fakeSave } from './api/fakeSave';

const btn = 'rounded-md border border-slate-300 bg-white px-4 py-2 text-sm font-medium shadow-sm
hover:bg-slate-100';

export default function App() {
  function save() {
    toast.promise(fakeSave(), {
      loading: 'Saving report…',
      success: (file) => `Saved ${file.name}`,
      error: (err) => err.message,
    });
  }

  function deleteItem() {
    toast('Item deleted', { action: { label: 'Undo', onClick: () => toast.info('Item restored') } });
  }

  function burst() {
    for (let i = 1; i <= 6; i++) toast.info(`Burst toast #${i}`);
  }

  return (
    <div className="min-h-screen bg-slate-50 text-slate-900">
      <header className="flex items-center justify-between border-b border-slate-200 bg-white px-6 py-
3">
        <h1 className="text-lg font-semibold">Toast &amp;amp; Notifications</h1>
        <NotificationCenter />
      </header>

      <main className="mx-auto grid max-w-md grid-cols-2 gap-3 p-6">
        <button className={btn} onClick={() => toast.success('Profile saved')}>Success</button>
        <button className={btn} onClick={() => toast.error('Payment failed. Try
again.')}>Error</button>
        <button className={btn} onClick={() => toast.warning('Storage almost full')}>Warning</button>
        <button className={btn} onClick={() => toast.info('New version available')}>Info</button>
        <button className={btn} onClick={save}>Promise (save)</button>
        <button className={btn} onClick={deleteItem}>Delete with Undo</button>
        <button className={btn} onClick={burst}>Burst ×6 (queue)</button>
        <button className={btn} onClick={() => toast.dismiss()}>Dismiss all</button>
      </main>
```

```
      <Toaster position="bottom-right" max={3} />
    </div>
  );
}
```

---

12. Wrap-up

Edge cases handled

- Burst of toasts (queue + cap), duplicate ids (upsert), promise lifecycle in one toast, close during loading.
- Pause on hover **and** focus with exact remaining time; progress bar in sync via `animation-play-state`.
- Exit animation without losing the element; idempotent dismiss; dismiss-all clears the queue.
- History: no duplicates, capped, persisted, safe when storage throws or is corrupted.
- A11y: always-mounted live region, `role="alert"` for errors, labelled buttons, `aria-expanded` on the bell, reduced motion.

Trade-offs to mention

Decision	Trade-off
Module-level store instead of Context	Callable anywhere, no Provider; but a singleton: two independent toasters on one page need a `createToastStore()` factory, and SSR needs per-request stores
One snapshot for toasts + notifications	Simple; `Toaster` re-renders when history changes. Fine at this size; a selector hook would fix it
Timer in `ToastItem`, not in the store	Queued toasts cost nothing and pause is local; but `toast.dismiss` timing depends on React being mounted
FIFO queue (oldest first)	Nothing is lost; but during a long burst the newest message waits. Alternative: drop/collapse oldest
History = toasts	Zero extra plumbing; a real product's notification feed usually comes from the server with its own ids and read state
`Date.now()` based `timeAgo` at render	Doesn't tick while the panel is open; fine for a demo

What to add with more time

- **Tests:** unit tests for `upsertToast`, `addNotification`, `timeAgo` (pure), and RTL + fake timers for pause/resume and the promise flow.
- **Swipe to dismiss** on touch, **stacked/collapsed** look (Sonner style) with expand on hover.
- **Pause all** when the tab is hidden (`visibilitychange`) or the window loses focus.
- **Cross-tab sync** of history via the `storage` event or `BroadcastChannel`.
- **Server notifications**: websocket/SSE feed, pagination of history, read state synced to the backend.
- **Custom content** (`toast.custom(jsx)`), per-call position, theming via CSS variables.

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just keep an array in `App` state and pass `addToast` down as a prop?

It works for a demo but every caller must be a component under `App`, and every toast re-renders the whole app. Toasts are a cross-cutting concern (API errors, sockets), so they need a global entry point. A small store with `useSyncExternalStore` gives that and re-renders only subscribers.

Q2. What is the difference between a toast and a notification here?

A toast is **transient**: shown for a few seconds, then gone. A notification is **persistent**: kept in history with read/unread state so the user can come back to it. Here every final toast also becomes a notification, so a toast the user missed is not lost.

Q3. What if product says "max 3 visible, drop the oldest instead of queueing"?

Change one place: `showToast` dismisses `toasts[0]` when `toasts.length >= max` before appending. The derived `slice` stays. I'd ask which matters more: never losing a message (queue) or always seeing the latest (drop).

State design

Q4. Why is `remaining` a `useRef` and not state?

It changes on every pause/resume but nothing on screen depends on its value; the visual progress is the CSS animation. Storing it in state would re-render for no visible change and could re-trigger the timer effect.

Q5. Why not store `unreadCount` or a `queue` array?

Both are derived: `filter(!read).length` and `slice(max)`. Storing them means two sources of truth that can drift (e.g. clear history but forget to reset the counter). Deriving is O(n) with n ≤ 50, which is nothing.

Q6. Why `useSyncExternalStore` instead of `useState` + `useEffect(subscribe)`?

The `useEffect` version can miss an update that happens between render and subscribe, and in concurrent rendering different components can read different versions (tearing). `useSyncExternalStore` is React's official API for external stores and handles both.

Challenge 1: global API

Q7. How would you support two independent toasters (e.g. app + an embedded widget)?

Turn the module into a factory: `createToastStore()` returns `{ subscribe, getSnapshot, toast }`, and `<Toaster store={...} />` takes it as a prop or via Context. The default export stays a singleton for the common case.

Q8. Does this work with SSR (Next.js)?

The module singleton would be shared across requests on the server, which is wrong. Toasts are client-only, so render `<Toaster />` in a client component and pass `getServerSnapshot` (empty state) to `useSyncExternalStore`; guard `localStorage` (already in `try/catch`).

Challenge 2: timers

Q9. Why not use `setInterval` every 100ms to update the progress bar?

That's 10 re-renders per second per toast for a purely visual effect. A CSS animation runs on the compositor (`transform: scaleX`) and `animation-play-state: paused` pauses it for free.

Q10. What happens when the user switches tabs?

Browsers throttle timers in background tabs, and the toast may expire unseen. With more time: listen to `visibilitychange` and pause all toasts while `document.hidden`. Since the notification history keeps the message, nothing is lost anyway.

Q11. StrictMode runs effects twice. Does the toast get dismissed twice?

No. The first effect run's cleanup clears its timer (and subtracts ~0ms), then the second run starts a new one. Only one timer is ever live. `dismissToast` is idempotent anyway.

Challenge 3: queue and promise

Q12. The user closes the "Saving..." toast before the promise settles. What happens?

The toast is removed. When the promise settles, `toast.success(msg, { id })` finds no toast with that id, so `upsertToast` appends a new one: the user still sees the result. If product wants "closed = silent", track dismissed ids and skip the update.

Q13. What if the API is slow (30s) or never resolves?

The loading toast is sticky by design, so it would stay forever. Wrap the promise with a timeout (`Promise.race` with a reject after N seconds) so it always ends in success or error, and keep the close button so the user can hide it.

Q14. What if 1,000 toasts are fired in a loop?

Only 3 render and only 3 timers run; the rest sit in an array (cheap). History is capped at 50. If even the array matters, cap the queue or collapse identical messages ("Upload failed ×12") using the id/dedupe path.

Challenge 4: animation and a11y

Q15. Why not wait for `onAnimationEnd` instead of a `setTimeout(EXIT_MS)`?

`animationend` never fires when animations are disabled (reduced motion, hidden tab, `display:none`), so the toast would never be removed. The timeout is guaranteed; the trade-off is that `EXIT_MS` must match the CSS duration.

Q16. How does a screen reader user know a toast appeared, and can they use the Undo?

The always-mounted `aria-live="polite"` region announces new toasts; errors use `role="alert"` to interrupt. The toast pauses while focused, so a keyboard user can Tab to Undo without a race. With more time: a shortcut (e.g. F8) to move focus into the toast region, like Sonner/Radix.

Q17. Why not trap focus in the toast like a modal?

Toasts must **not** steal focus: the user is in the middle of something else. They are announced, not focused. Only the notification panel is interactive on demand, and it closes on Escape / outside click.

Implementation steps

Q18. How do you test the timer logic?

Pure parts (`upsertToast` , `addNotification` , `timeAgo`) with plain unit tests. For `ToastItem` : React Testing Library + `vi.useFakeTimers()` ; render, advance 3s, `fireEvent.mouseEnter` , advance 10s → still there, `mouseLeave` , advance the remaining 1s → gone.

Q19. Why does a history id look like `t7:success` ?

So the same toast id updated to the same final type twice cannot create two rows, while a toast that changes type (e.g. retried and failed again with a new id) still gets its own row. It is a cheap idempotency key.

Performance and scaling

Q20. The whole history re-renders `Toaster` when a notification is marked read. Is that a problem?

At 3 toasts and 50 notifications, no. If it grew, add a selector hook: `useToastStore(s => s.toasts)` built on `useSyncExternalStore` with a selector, or split into two stores so each component only subscribes to what it reads.

Q21. How would you sync history across tabs or devices?

Across tabs: listen to the `storage` event (fires in other tabs on `setItem`) and replace `notifications` , or use `BroadcastChannel` . Across devices: the backend becomes the source of truth for notifications (ids, read state), fetched/paginated and pushed via websocket; toasts stay client-side.

Trade-offs

Q22. Why not just use `react-hot-toast` or `sonner` ?

In production I probably would. In this interview the point is to show the pattern those libraries use: an external store, an imperative API, update-by-id, pause-able timers and an accessible live region. Knowing the internals also helps when we need to customise them (notification history, server feed).

Q23. Why Tailwind class maps (`STYLES[type]`) instead of building class names like `bg-${color}-500` ?

Tailwind generates classes by scanning source text; dynamically built strings are never seen, so those classes would be missing in the build. Full literal class names in a map are always picked up.